The @timestamp_opts module attribute on line 4 tells Ecto that our timestamps should include microseconds (usec). Without this, our timestamps would only contain seconds. It can be helpful to have this higher fidelity timestamp when debugging a system, so I recommend that you always use utc_datetime_usec timestamps.

Use the Schema as a Struct

Ecto schemas are just structs, so we can use them with the %Struct{} syntax that you used earlier in this book. Let's try this out with the SMS Message schema. Start an IEx session with the -S mix option and try it out.

(You'll see a warning that the Contact schema doesn't exist, but this is safe to ignore for now. The warning comes from our association referencing a module that doesn't exist. If it becomes a problem and your project won't run, then create the Schema.Contact module found at the end of this section.)

```
$ iex -S mix
iex> alias PhoneApp.Conversations.Schema.SmsMessage
iex> message = %SmsMessage{to: "+1-111-222-3333"}
%PhoneApp.Conversations.Schema.SmsMessage{
  __meta__: #Ecto.Schema.Metadata<:built, "sms_messages">,
  id: nil,
  contact_id: nil,
  contact: #Ecto.Association.NotLoaded<association :contact is not loaded>,
  ...
  to: "+1-111-222-3333",
  ...
}

iex> message.to
"+1-111-222-3333"
```

Because schemas are mapped to structs, you can use most of the same functions and operations that you are used to on structs. (Ecto doesn't implement the Access protocol for schemas, so some features are not available.) For example, you can build a struct using the %{} syntax, and you can modify fields using the Map.put/3 function.

We'll come back to the contact association in a moment, but let's talk about fields first.

Schema Fields

The field/3 macro function has many different options that you can use to define a schema the way you want. We'll cover a few of the different options, but the most important is the field type.